

Developing strategies for the Crypto Market

Predicting the price of crypto coins using momentum and exponential moving average strategies

David Jackson

1. Introduction Predicting crypto markets is difficult because there is not the same information as there is for stocks. The price of a stock of a commodity or service may reflect the price of other stocks. For example, the price of many petroleum-based products may reflect the price of petroleum oil. Another example may be the price that Ford trades at. If rubber is scarce and the price is driven up, then the price of tires will be driven up, which will increase the price of a car with the tires. What is being described here is the efficient market hypothesis, which states that stocks trade at a price that reflects all of the available information about that stock. This means that trading stocks can be easier because if you have all of the available information, you should know what the stock price should be, and if it differs from what it should be, you can buy it or sell it. But Academic research largely indicates that cryptocurrency markets are inefficient. This inefficiency means that prices don't always reflect all available information, creating potential opportunities for investors to generate returns. Crypto markets are also extremely volatile. Due to this extreme volatility, predicting the markets can be hard, but it can also give opportunities to make returns. Unlike stocks, which are tied to company performance and cash flow, or commodities with industrial uses, most cryptocurrencies lack a clear intrinsic value. This makes it difficult to apply traditional financial analysis, causing prices to be heavily driven by speculation, social media hype, and market sentiment rather than fundamentals.

2. Collecting and Cleaning the Data Before we get data from Binance or YFinance, we will develop our strategy using randomly generated data. The reason we will do this is so that we can generate random data that has an average return of zero. The primary purpose of having a average return of zero, essentially means if you invested in this stock you would not make or lose money over time. This begs the question though how are we supposed to make money on a stock that has zero returns. This is where our strategies come into play, which will give us an indicator of when to buy and when to sell, so we can take advantage of the variance in our artificial data to make returns. After this, we will collect the real data and implement our strategies on it to see how we do.

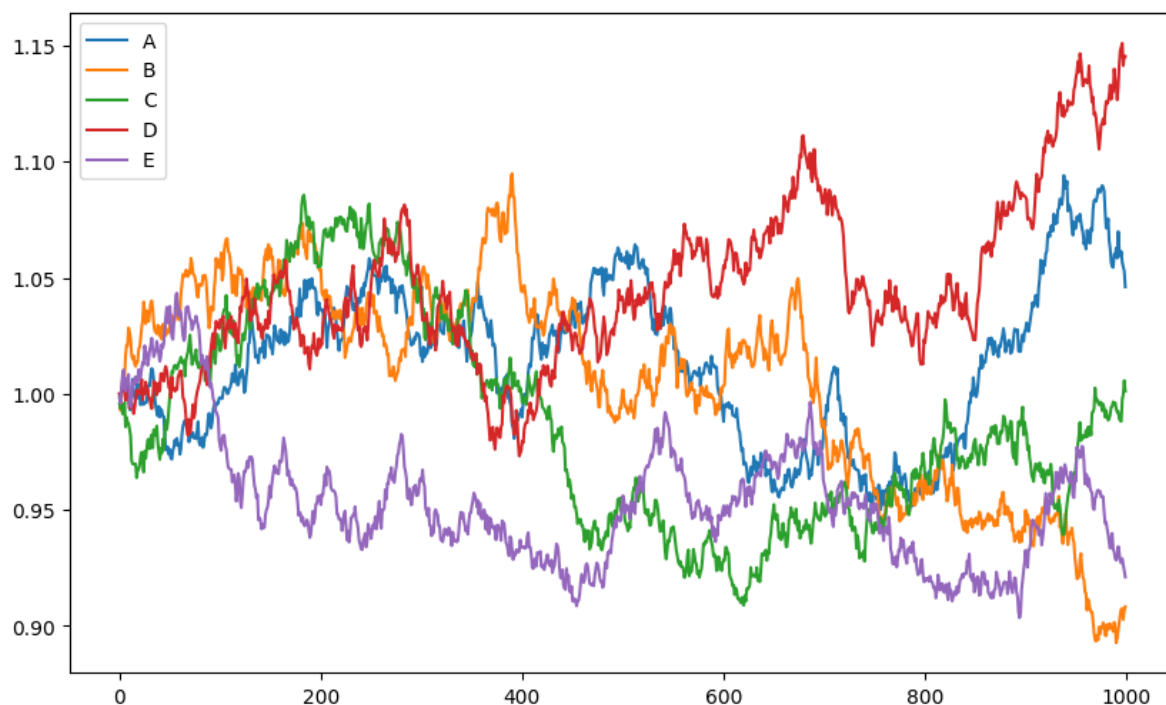


FIGURE 0.0.1. Random data returns

3. strategy The first strategy we developed was the momentum strategy, but before we moved to the momentum strategy, we did some exploratory data analysis and confirmed that the artificial data is correct. Here is the random data returns over time. We can consider this data to be on any time frame we want; it could be minute data, hour data, daily data, or yearly data. So, let's go with daily scale, as we can see that as the days increase to 1,000, the stocks returns stay around one which means we are not making any profit. Also, note that before continuing, we are considering these to be simple returns, although they are randomly generated, because when we use actual coin data, we will calculate the returns using simple returns rather than log returns.

Note that the coins are just named A, B, C, D, and E. These are the names of the coins with their returns over time on the y-axis. The next thing we do is look at the Sharpe ratio for each of the individual stocks in the graph below. We can see that some of the stocks have negative Sharpe ratios, and some of them have positive ones. But overall, none of them have a good Sharpe ratio and would not be a good investment to buy and hold, which makes sense since there are zero returns, which is how we design the data.

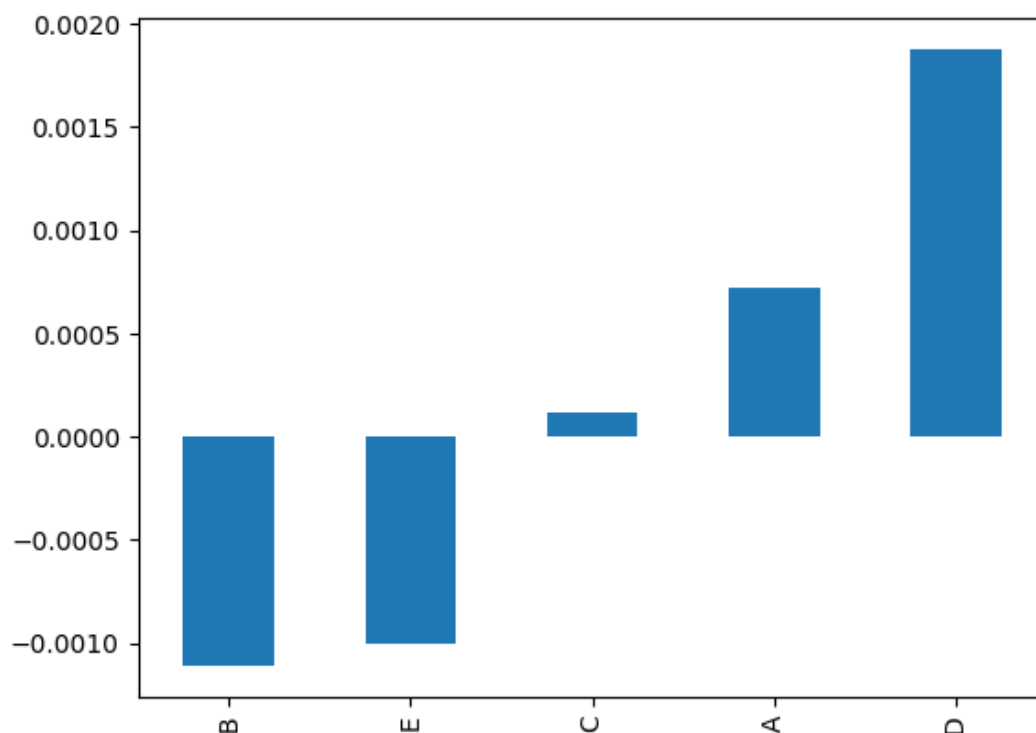


FIGURE 0.0.2. Sharpes ratio for random data

To make our momentum strategy, we need to generate price data. More specifically, we need the open, high, low, and close data for the stocks, which we will generate like the returns data. Once we have this data, we will make our strategy by defining a function that will identify breakouts, which would indicate momentum in that stock. A long entry will happen when we hit a high greater than the past y days high, where y is a variable of the look-back period. The long exit is defined when the low is less than the past x days low, where x is the variable

look-back period. The short entry is defined as a low that is less than the past n days low, where n is the variable look back. The short exit is defined as when a high is greater than the previous m hours days, where m is the variable look back.

	A				B			
	Open	High	Low	Close	Open	High	Low	Close
0	102.101453	87.344192	108.167198	83.549694	92.369042	95.431629	104.313264	94.420836
1	89.303885	90.359143	99.068323	96.336677	104.671806	95.671177	96.258002	97.489150
2	112.392284	105.846373	97.666164	106.236753	97.164388	94.913888	96.866597	115.948787
3	101.814156	94.492466	108.933444	99.811214	103.470808	92.644610	95.906925	95.709992
4	106.500917	89.595711	99.491702	95.342966	93.786588	100.421908	92.047309	104.420481
...	...	...	...	...	...	...	...	...
995	104.062229	91.709838	102.869590	100.043879	94.587564	91.950209	103.720753	93.554894
996	89.155388	100.550368	103.888953	85.007238	88.098797	86.791869	102.213019	92.026207
997	107.643866	88.433023	111.467342	106.646375	95.939602	99.014099	90.787796	109.938515
998	99.802456	115.312094	86.682233	97.853227	85.742221	91.702465	94.946640	104.208036
999	91.984750	91.844660	100.304398	98.370126	114.134467	104.854285	109.507514	101.866713

1000 rows × 20 columns

FIGURE 0.0.3. Data for Open, High, Low, and, Close

Now that we have our function that returns a data frame of positions, 1 for a long entry, -1 for a short entry, and 0 for doing nothing. We will also get optimal weights for our stocks using the Sharpe ratio. This will tell us what percent to invest in each coin. Now that we have our positions data frame, our returns data frame, and our weights. We multiply them together to get strategy returns, which we can plot to see how the different coins did with this strategy. When plotting will use the cumulative product since we are using simple returns. This is because simple returns we have to account for compounding, that's why it is a product, whereas log returns, you can just use a cumulative sum.

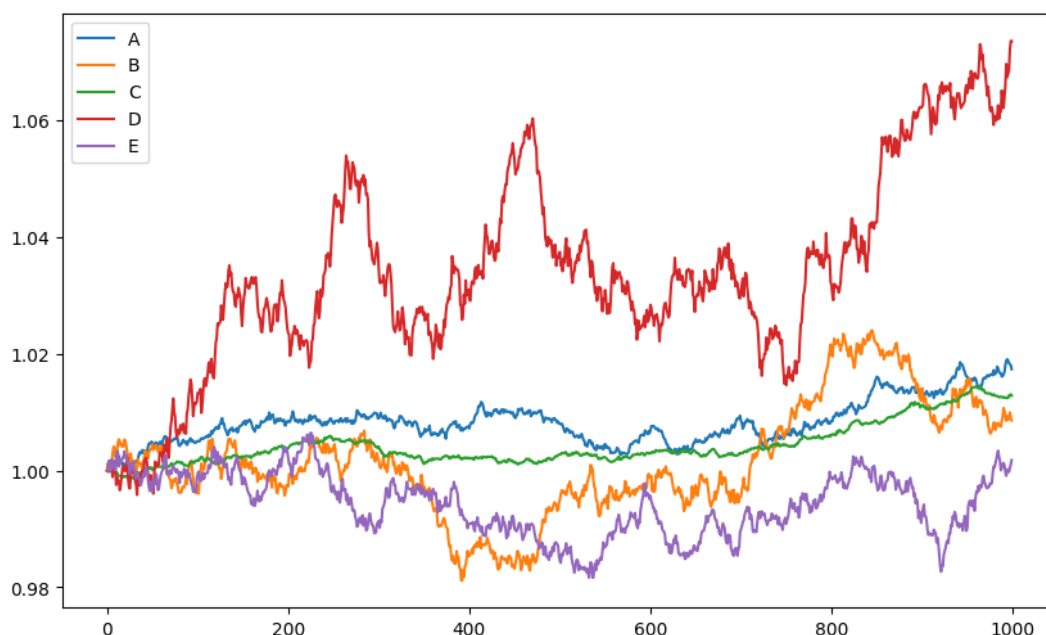


FIGURE 0.0.4. momentum strategy returns

Looking at the coins for our strategies, we can see that D did very well, and A, B, and C did okay, and E lost money at the beginning and then gained it back at the end, balancing out. We also calculated the Sharpe ratio for each of these coins: A: 0.703669, B: 0.155227, C: 1.183971, D: 0.727880, E: 0.038341. We do know that all of the Sharpe ratios are positive, which is better than what we started with, because some of them are negative. This implies that our strategy is working and has improved our Sharpe ratio, although the Sharpe ratios aren't as high as we would like, considering that our data has zero returns, this strategy did very well. The next thing we did is plot the portfolio returns. This is all the stocks combined into one overall plot to see how they did combined. As we can see, the portfolio overall did very well, and had a Sharpe ratio of 0.8696951715281144. Which is not great. Anything above 2 is usually considered good. The next thing we will do is move on to trying to identify the optimal look-back period, to see if we can get a better Sharpe ratio for our momentum strategy.

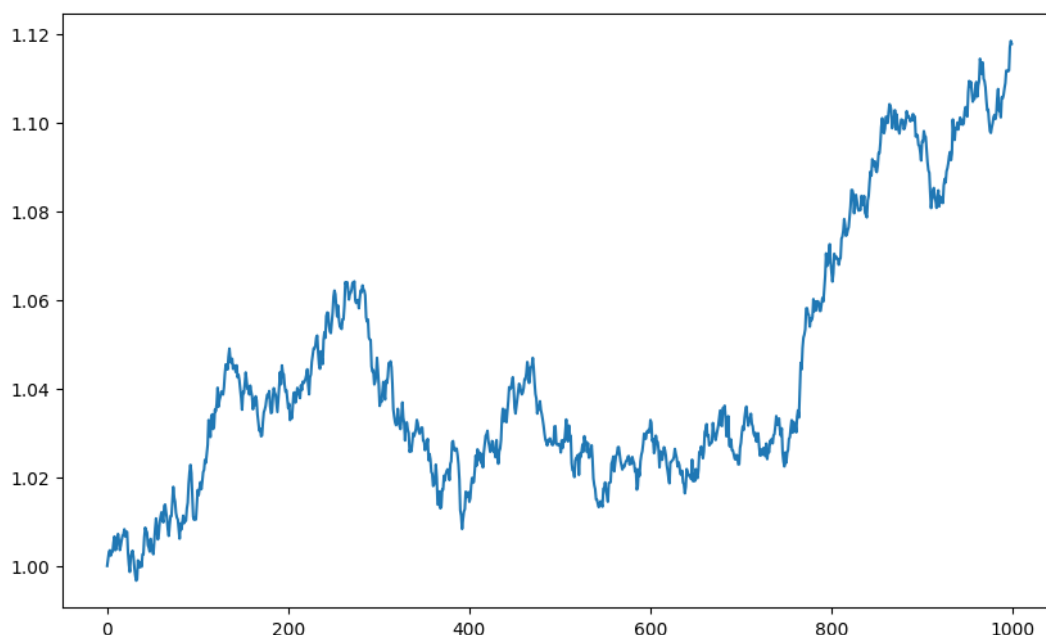


FIGURE 0.0.5. portfolio returns

To find the optimal look back periods, we will use a heat map that will show areas with the highest Sharpe ratio. We have to do this individually for a long position and individually for a short position, and it only works for one coin at a time. To do this, we need to define a new function that just looks for breakout long and one for breakout short. These are defined the same way as our momentum function; it's just split up into two functions for the two different graphs. Looking at the graphs below, we can see that the optimal Sharpe ratio for a long entry is probably between 20 and 25 for the Y variable and three for the X variable. For the short entries, there isn't really a good optimal spot with a high Sharpe ratio, and it doesn't really line up with the long entries, which will make it hard to choose a good look back. I did this with all of the different coins, A through E. A is the one that is given in the graph. I will not provide all the graphs here, but another problem that we run into is that the optimal look-back periods for the individual coins do not line up with each other, so there isn't exactly one look-back we can choose what will work for all of the coins.

This is also the same for the short side, although we can make the lookback for the long and the short side different when we apply the momentum strategy; it does it to the whole portfolio of coins, so we don't have individual lookback periods for individual coins. So there wasn't one look back that we could use. After trying out a bunch of different look-backs, we found that looking back one day for all of the variables works the best

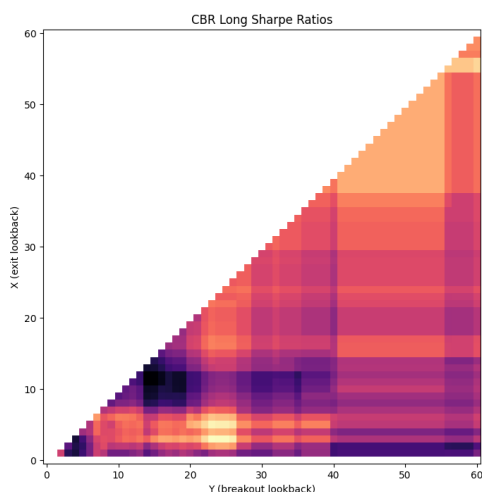


FIGURE 0.0.6. heat map long

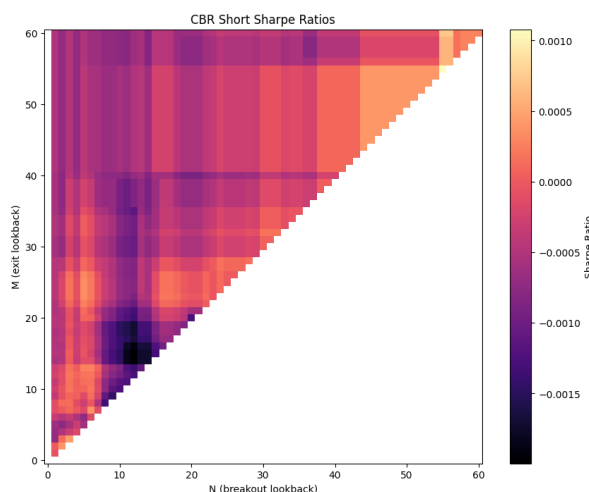


FIGURE 0.0.7. heat map short

FIGURE 0.0.8.

We will come back to the momentum strategy with real data later. Still, for now, we will move on to the exponential moving average strategy with artificial data, and we will also do this with real data later.

The general goal of the exponential moving average strategy is to figure out when the price is falling or rising. When you have two moving averages, when the one with the shorter look back is above the one with the longer look back. That should tell you that the price is rising, and then when they cross over, that tells you that you should sell short, and you just keep doing this back and forth to gain from this price movement over time. The difficulty in this strategy is trying to find a good look back. Because the variation in the price is random from the randomly generated data and also has zero returns, trying to find an exponential moving

average that will ignore the noise in all the small price changes and see the overall price swings was difficult. We ended up going with a look back for the first exponential moving average of 64, 34, and for the second one. After we get this, we generate a signal. This is just found by taking the difference between the 34 exponential average and the 64 exponential moving average. If this value is positive, we should buy; if it is negative, we should sell. Then, once we have this, we can multiply it by the returns and see how well our strategy does. Here is the graph of the returns for the individual stocks and the portfolio.

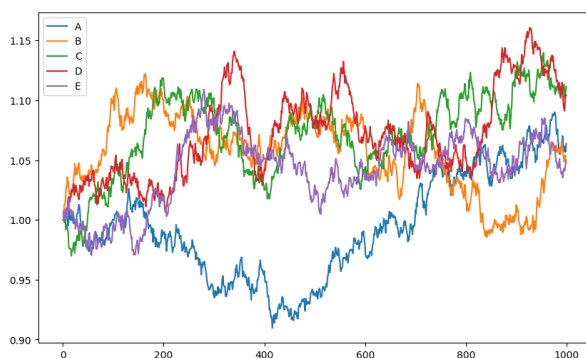


FIGURE 0.0.9. returns for
EMA all stocks

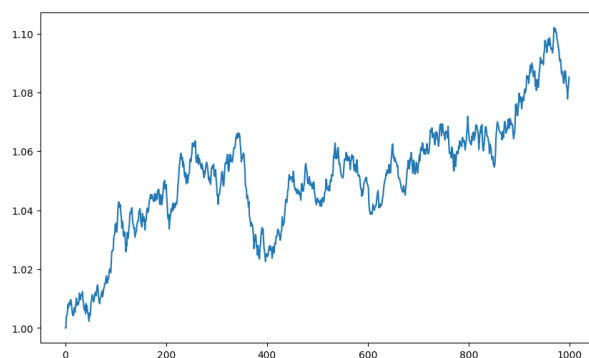


FIGURE 0.0.10. returns for
EMA combined

FIGURE 0.0.11.

Looking at the returns, we can see that the strategy performed well, and we had a Sharpe ratio of 0.002439 for the overall portfolio of coins, and the independent coins did well. So overall, this strategy did okay but not great, and we will move on to performing this with real data. We will use Binance to get our data, and we have about 12 different securities we are testing these strategies on. After importing all the data we needed to calculate the returns, we did that, then we could plot those over time to see how these crypto coins would have performed over time. We use data from 2022 to 2026, and we are on the 4-Hour time chart. Here is the graph of them.

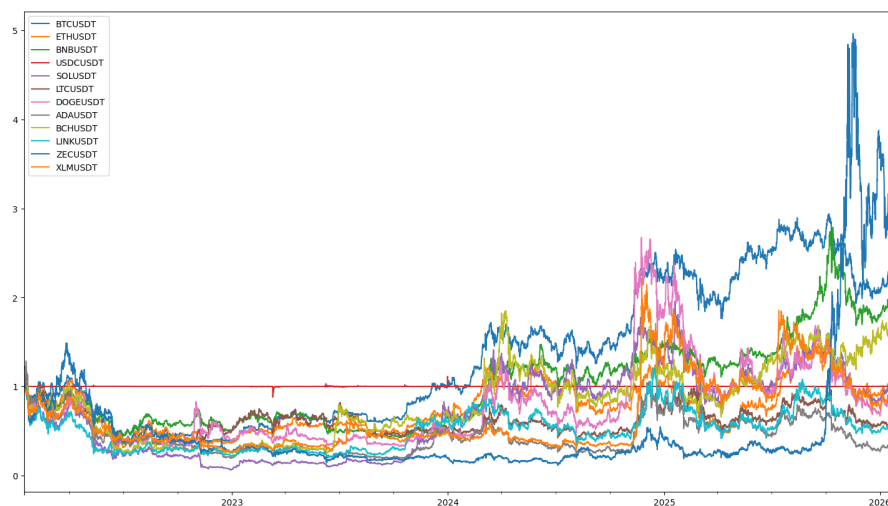


FIGURE 0.0.12.

After this, we computed some basic statistics like the mean and standard deviation, but the main thing we did was find the Sharpe ratio of each of these coins and plotted them to see which ones perform the best.

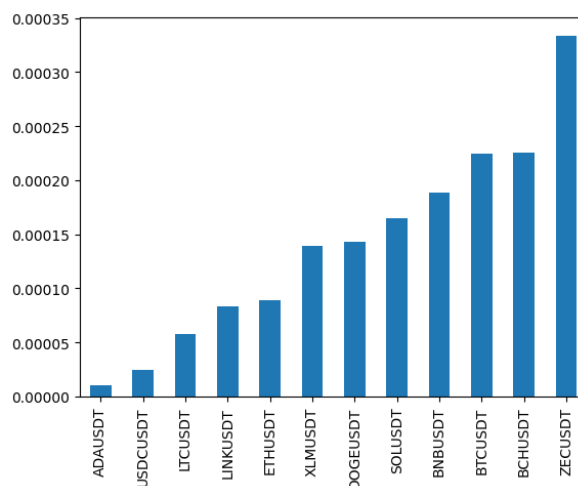


FIGURE 0.0.13.

After this, we found the optimal weights using the Sharpes ratio to determine what percent of our portfolio we should invest in each of these coins. Then, once we had finished that, we moved on to applying the momentum breakout strategy to this portfolio to get our signal data

frame. Which tells us when to buy and when to sell. Then we can multiply our positions by a returns by our weights and plot the strategies returns. The following graph is a plot of all the individual securities and their returns. Beside that plot is the plot of the portfolio returns. This is just all of them combined into one.

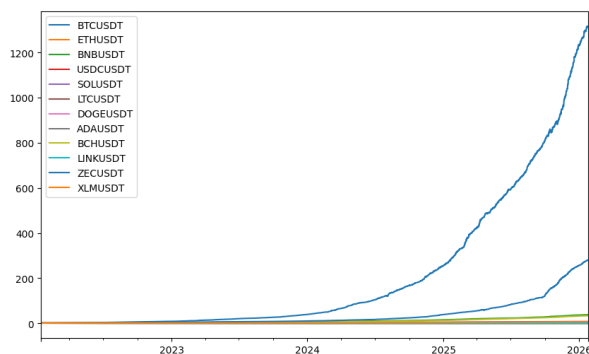


FIGURE 0.0.14. strategy re-
turns for coins

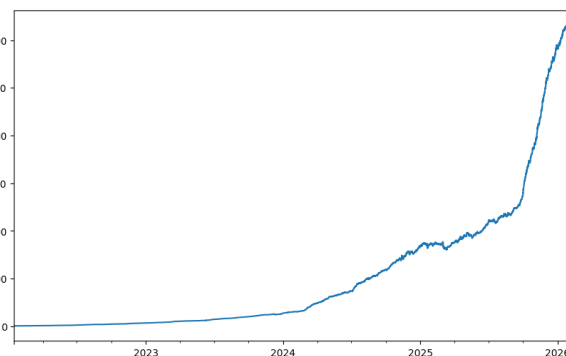


FIGURE 0.0.15. strategy returns

FIGURE 0.0.16.

As we can see from looking at the graphs, the momentum breakout strategy performed very well, but the Sharpe ratio of the portfolio return was 0.004, which is not very good. The Sharpe ratio of the individual coins varied; some of them were positive, and some were negative. Some of the coins performed well, some of them didn't, but overall the portfolio performed well. Now, we will move on to applying the exponential moving average strategy to the real data. After applying the exponential moving average strategy, the best look-back period is 132 and 66 for them. After that, we got our signals data frame, and then we're able to get the returns for all the individual coins in the following graph, given below, and we can also combine them into one portfolio to get the graph given below. Looking at them, we can see that some of the coins performed badly, and some of them performed well. The overall portfolio performed pretty good, but the Sharpe ratio was only 0.0004, which isn't the best.

We expected this to do better than what it did, but considering that the volatility for the randomly generated data was $\frac{0.075}{\sqrt{365}} = 0.004$ and the average volatility across the coins for the real data was 0.3193975, which is a lot more volatility than the random data. This is probably why this did not do as well. Due to the exponential moving averages not being able to ignore the smaller price movements or noise. To be able to pick up on the bigger price movements, to take advantage of the swings in price. What was probably happening was that it picked up on the smaller price movements and bought or sold, but then the price immediately swung the opposite way, which caused it to lose money before the signal could pick up on the price movement. Once it picked up on the price movement, it probably switched, in which the price went back the opposite way before it could pick up on it, causing it to lose money on the small price swings, but it still made money overall. To avoid some of these smaller price movements, we would need to increase the lookback, but we tried that, and it resulted in a smaller Sharpe ratio. This is probably because of the longer look back; the strategy cannot react as quickly, which would cause us to lose out on some of the medium price swings and not get the full profit from the large swings.

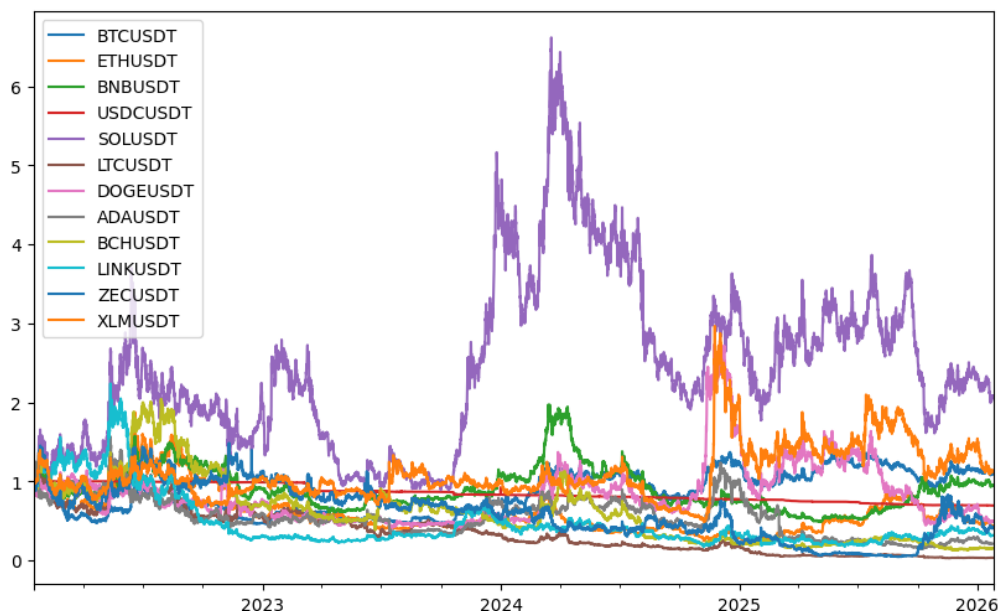


FIGURE 0.0.17.

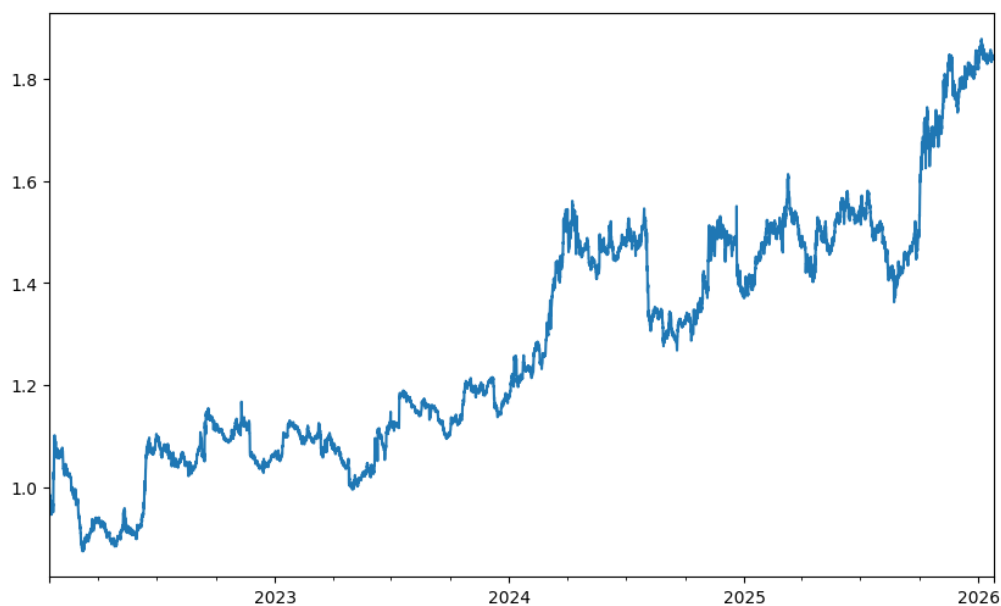


FIGURE 0.0.18.

4. Results

The results of this project demonstrate that both the momentum breakout strategy and the exponential moving average (EMA) strategy are capable of generating positive risk-adjusted returns in cryptocurrency markets, particularly when applied at the portfolio level. This was true for both artificially generated data and real cryptocurrency price data, though performance differed across strategies and datasets.

When applied to the artificially generated data with zero expected returns, the momentum breakout strategy produced positive Sharpe ratios for all five synthetic assets. While none of the individual Sharpe ratios were exceptionally large, they were all improvements over the negative or near-zero Sharpe ratios observed under a buy-and-hold approach.

This indicates that the momentum strategy was able to exploit short-term price movements and variance in the data despite the absence of a positive expected return. When these assets were combined into a portfolio using Sharpe-ratio-based weights, the strategy performed substantially better.

This suggests that diversification across assets significantly improved performance and reduced risk. The analysis of optimal look-back periods using heat maps revealed that there was no single set of parameters that maximized performance across all assets simultaneously. Optimal look-back windows differed not only between assets but also between long and short positions. This result highlights a practical limitation of systematic momentum strategies: parameter choices that work well for one asset may not generalize well to others. Despite this, a uniform look-back period of approximately one day produced strong overall results and was selected as a reasonable compromise.

The EMA crossover strategy also performed well on the artificial data. Using look-back windows of 34 and 64 periods. This indicates that even in a market with no inherent trend or expected return, technical indicators can sometimes extract value by identifying persistent price movements. However, the EMA strategy appeared more sensitive to parameter choice and noise than the momentum breakout strategy.

When applied to real cryptocurrency data from Binance over the 2022–2026 period on a four-hour time frame, the momentum breakout strategy performed exceptionally well. While individual asset performance varied widely—with some coins producing negative Sharpe ratios the portfolio as a whole achieved a Sharpe ratio of approximately 0.004. This result suggests that the momentum breakout strategy was particularly effective in capturing large price trends present in cryptocurrency markets during the sample period. The portfolio performance further supports the idea that crypto markets exhibit inefficiencies that can be exploited using systematic trading strategies.

In contrast, the EMA strategy applied to real data produced more modest results. Although some assets performed well, the overall portfolio Sharpe ratio was approximately 0.0004. While still positive, this performance was significantly weaker than that of the momentum breakout strategy. This difference suggests that simple EMA crossover strategies may struggle in highly volatile markets where price movements are abrupt and trend persistence is inconsistent.

5. Conclusion

Part one of this project investigated the effectiveness of momentum breakout and exponential moving average strategies in cryptocurrency markets using both artificial and real price data. The results provide evidence that technical trading strategies, particularly momentum-based approaches, can generate positive risk-adjusted returns in crypto markets, supporting the hypothesis that these markets are inefficient.

One of the most important findings is that portfolio construction plays a critical role in strategy performance. While individual assets often exhibited inconsistent or weak performance, combining them into a diversified portfolio using Sharpe ratio based weights significantly improved results. This was especially evident in the real data momentum strategy, where strong portfolio performance was achieved despite mixed individual outcomes.

The comparison between artificial and real data also highlights the importance of market structure. While both strategies were able to extract value from artificial data with zero expected returns, performance was substantially stronger when applied to real cryptocurrency markets. This suggests that real crypto price dynamics contain exploitable patterns such as trends and breakouts that are not fully captured by random processes.

However, the project also reveals several limitations. Strategy performance was highly sensitive to parameter choices, particularly look-back periods, and no single parameter set worked optimally across all assets. Additionally, the strategies did not account for transaction costs, slippage, or liquidity constraints, which could materially reduce real-world profitability. The extremely high Sharpe ratio observed for the momentum strategy may therefore overstate achievable performance in practice.

In part 2 of this project, we extend this analysis by incorporating transaction costs and slippage, Alpha and Beta testing, and Regression models for testing. Despite these limitations, the results of this project suggest that momentum based strategies are particularly well suited to cryptocurrency markets and provide strong evidence that systematic trading approaches can exploit market inefficiencies in this asset class.

Part 2 Transaction costs and models

1. Artificial Data

We were using weights to find what percentage we should invest in each coin, but this was only calculated once. This is okay for our earlier data analysis, making it simpler, but it is not the best to hold the same percentage of a coin in your portfolio over time. We want to rebalance the portfolio, changing the percentage of each coin in that portfolio over time. To do this, we need to have our code recalculate the weights. To do this with our artificial data, we will just rebalance it about every 3 months, which would be 90 data points. So we make a window that will recalculate the weight every 90 days. At the same time, we will use the same 90-day window for our returns and positions, multiply them together to get our strategy return, and then combine them to make a portfolio. Then we can plot our strategy's returns over time and see how it performs. This first plot is for our momentum breakout strategy. We will do the exponential moving average later. This is also with our artificial data, and we will move to our live data later. Below is the graph of our strategy returns, rebalancing our weights every 90 days. If you want to see what it looked like before we were using the rebalancing, refer to Figure 0.0.5, which plots what returns look like when only calculating weights one time for the portfolio. Comparing the two, we see we end at about the same percent return, somewhere around 12 percent return, but in Figure 5, it is much more volatile and does not have a stable of growth compared to rebalancing every 90 days which grows much more steadily in an upward trend whereas before we moved horizontally for a while before shooting up closer to the end of the trading. This shows that rebalancing our portfolio is working, although it is not generating us more returns than calculating it one time; it is reducing our risk and giving us more steady growth.

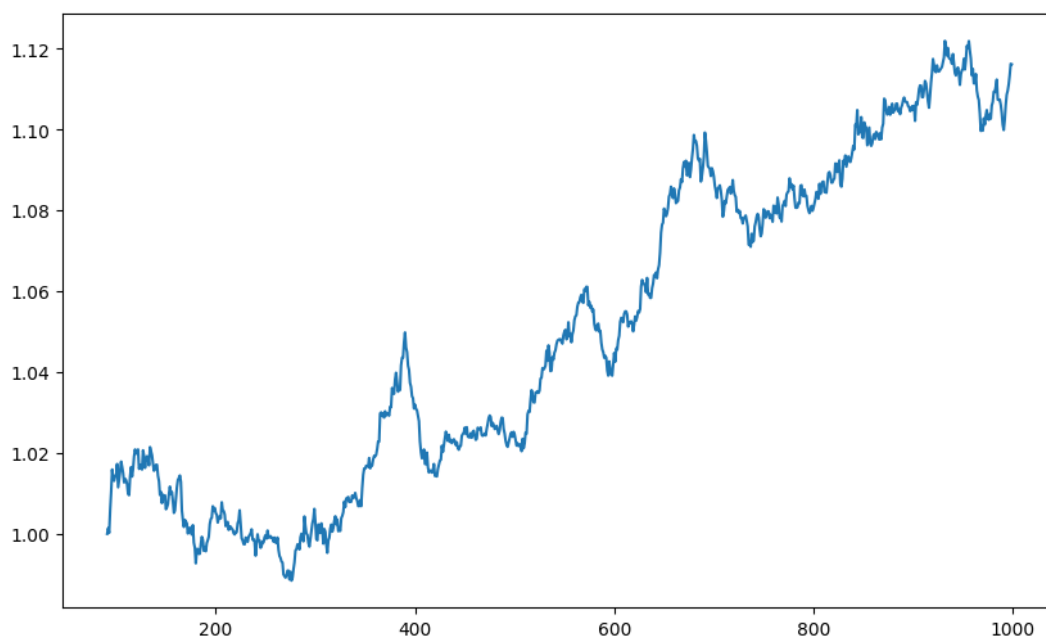


FIGURE 0.0.19.

The next step is to add transaction costs and slippage. Although these can be calculated, it can be rather complicated to calculate how we calculated. So we will assume that there will be about a 7 basis point transaction cost, 13 basis points of slippage, for a total of 20 basis points. To add this into our portfolio we need to find the turnover which is how many times we are trading once we have that we can take our returns and subtract it from the turnover multiplied by our 20 basis points of slippage and transaction costs to get our net returns then we can plot our net returns to see how our strategy performs with t cost and slippage. Looking below at our plot, we can see that we basically steadily lose money with this strategy after accounting for transaction costs, which essentially means that our strategy is not generating enough returns to offset the cost of the strategy. This could be due to the strategy trading too often, which is likely the case because of the look-back. The look-back is rather small. The reason we went with a smaller look back is that when we found the heat map earlier,

It showed that the optimal look back was small, and this seemed to be the case because we tested a bunch of different look backs. When using a smaller look back, we generated more returns, but this also increases the t costs. We also found this to be the case for the real data, but that will be explained later. Obviously, this is not good. Our strategy is not profitable when accounting for t costs and slippage. This is our artificially generated data, that is designed to have zero returns, so it is not unexpected that our strategy was not successful with the artificial data. It is somewhat expected that this would happen when accounting for transaction costs.

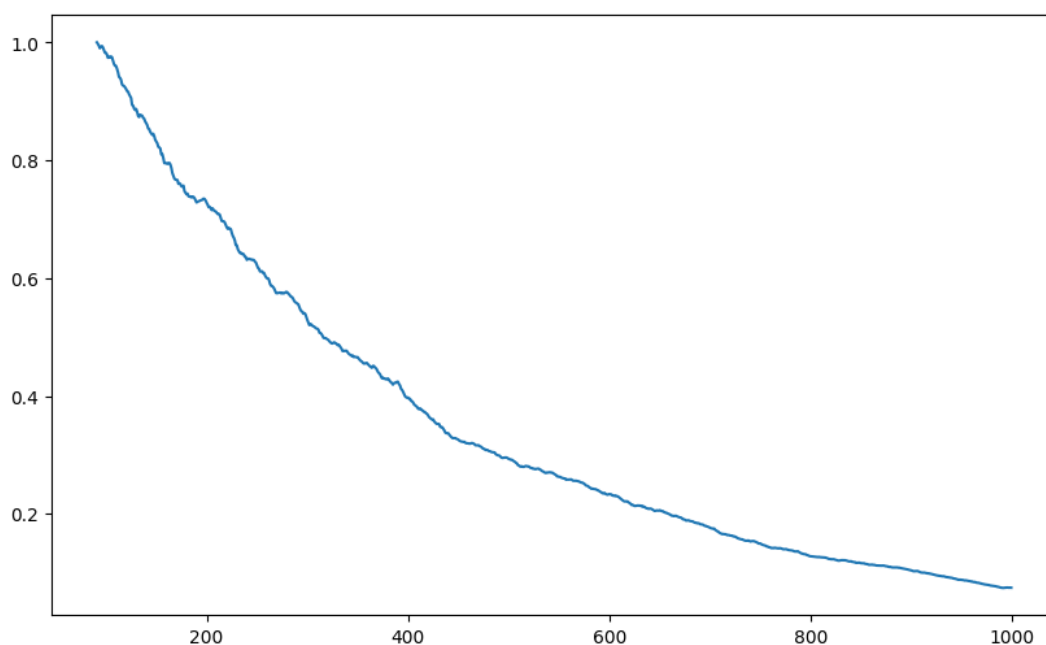


FIGURE 0.0.20.

The next step is to generate a regression model to calculate our Alpha and Beta to see how well our strategy performs compared to a benchmark. For our benchmark, we will just be using the average returns across all the coins for the artificial data. For the real data, we will use BTC as the benchmark. Given below is the regression of our strategy compared to the benchmark. Alpha is the constant term plus the error term, and Beta is the coefficient of the benchmark.

$$Strategy_t = \beta \times SPY_t + \alpha + \epsilon_t$$

Beta Contribution
Alpha Contribution

FIGURE 0.0.21.

OLS Regression Results						
=====						
Dep. Variable:	y	R-squared:	0.044			
Model:	OLS	Adj. R-squared:	0.043			
Method:	Least Squares	F-statistic:	33.68			
Date:	Tue, 17 Mar 2026	Prob (F-statistic):	9.72e-09			
Time:	16:09:07	Log-Likelihood:	3551.2			
No. Observations:	727	AIC:	-7098.			
Df Residuals:	725	BIC:	-7089.			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	0.0001	6.8e-05	1.872	0.062	-6.21e-06	0.000
0	0.2169	0.037	5.803	0.000	0.144	0.290
=====						
Omnibus:	11.966	Durbin-Watson:	1.997			
Prob(Omnibus):	0.003	Jarque-Bera (JB):	18.529			
Skew:	0.107	Prob(JB):	9.47e-05			
Kurtosis:	3.752	Cond. No.	550.			
=====						

FIGURE 0.0.22.

Looking at the Alpha term, it is very low, which indicates that our strategy is not beating the market by much at all. At first, this would seem to be strange because our benchmark has an average return of zero, because that is how we generated our artificial data, so why is our Alpha so low? It is low because profits are coming from co-movement with the market and not a pure independent edge. The Beta indicates that when the benchmark moves by 1%, the strategy moves by 0.22% on average. This indicates that our strategy is less sensitive than the market. A low Beta for our strategy isn't necessarily bad because we are trying to take advantage of swings in the market. So it makes sense that our strategy is correlated with the market.

Note that this regression is for our returns before accounting for transaction costs and slippage. This is also just the training model, which is 80% of the data. We also did a model for our test data, which is the other 20% of the data, and it showed very similar results with a very low Alpha and a Beta of 0.3. We will not bother with making a model after the transaction costs are accounted for because we have already seen that the transaction costs make this strategy unprofitable, so we can already assume that making a model after accounting for transaction costs will have worse results than the model we already made.

What does all of this mean? The Beta indicates that our strategy is taking advantage of Market moves, but only partially. This means we are exposed to less risk, but are not generating good returns. The Alpha indicates that we have little to no edge on the market, which, for our strategy, is not necessarily a bad thing, but when we look at the T-stat, we see that $1.87 < 2$. This indicates that we are not generating returns that are significantly different from zero after adjusting for Beta this implies that any edge we have is weak and not significant. Since our returns are not statistically significant, we didn't bother with a model that accounted for transaction costs; we would just see even worse results in the model. Although these results are not good, they're not unexpected for the artificial data, and now we will move on to the exponential moving average strategy.

Moving on to the exponential moving average strategy, we do the same thing and create a window in which weights will be calculated and rebalanced about every 90 days. Below is a graph of the returns of the exponential moving average strategy before accounting for transaction costs and slippage. We can see that the strategy performs well before accounting for t costs, but after we add the costs, the graph looks something like this.

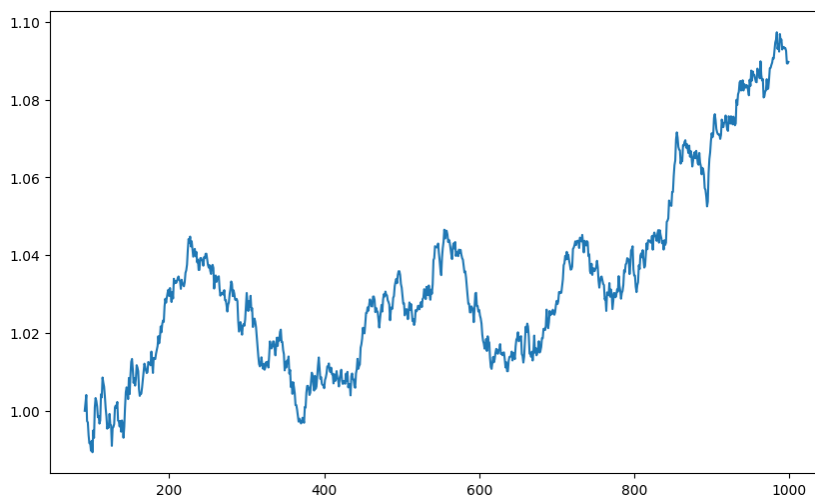


FIGURE 0.0.23. Before T Costs

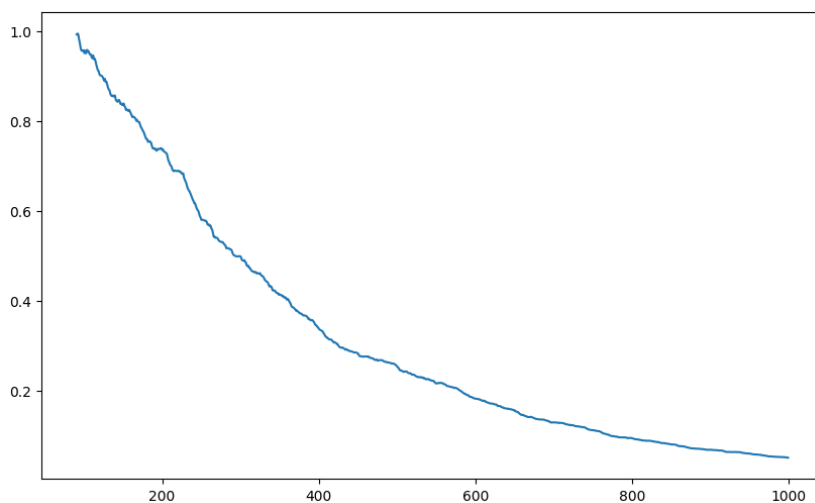


FIGURE 0.0.24. After T Costs

As we can see, after transaction cost are strategy is not profitable and steadily loses money over time. This indicates that our strategy is not generating enough returns to offset the cost of the strategy. This could also indicate that it is trading too frequently, causing it to generate transaction costs. This is not unexpected with our artificial data, though it is very common to have a strategy fail when using artificial data after accounting for transaction costs. Although our strategy is not profitable, we will move on to generating a model to find our Alpha and Beta, which should give us some insight into what is going on.

OLS Regression Results							OLS Regression Results						
Dep. Variable:	y	R-squared:	0.002	Dep. Variable:	y	R-squared:	0.030						
Model:	OLS	Adj. R-squared:	0.000	Model:	OLS	Adj. R-squared:	0.025						
Method:	Least Squares	F-statistic:	1.129	Method:	Least Squares	F-statistic:	5.547						
Date:	Tue, 17 Mar 2026	Prob (F-statistic):	0.288	Date:	Tue, 17 Mar 2026	Prob (F-statistic):	0.0196						
Time:	17:27:36	Log-Likelihood:	3470.4	Time:	17:27:40	Log-Likelihood:	879.38						
No. Observations:	727	AIC:	-6955.	No. Observations:	182	AIC:	-1755.						
Df Residuals:	725	BIC:	-6946.	Df Residuals:	180	BIC:	-1748.						
Df Model:	1			Df Model:	1								
Covariance Type:	nonrobust			Covariance Type:	nonrobust								
	coef	std err	t	P> t	[0.025	0.975]		coef	std err	t	P> t	[0.025	0.975]
const	5.629e-05	7.5e-05	0.750	0.453	-9.1e-05	0.000	const	0.0002	0.000	1.542	0.125	-6.22e-05	0.001
0	-0.0438	0.041	-1.063	0.288	-0.125	0.037	0	0.1812	0.077	2.355	0.020	0.029	0.333
Omnibus:	4.074	Durbin-Watson:	2.154	Omnibus:	7.984	Durbin-Watson:	2.242						
Prob(Omnibus):	0.130	Jarque-Bera (JB):	4.067	Prob(Omnibus):	0.018	Jarque-Bera (JB):	7.887						
Skew:	0.129	Prob(JB):	0.131	Skew:	0.502	Prob(JB):	0.0194						
Kurtosis:	3.260	Cond. No.	550.	Kurtosis:	3.184	Cond. No.	535.						

FIGURE 0.0.25. Training Set

FIGURE 0.0.26. Testing Set

FIGURE 0.0.27.

Above are the training set and test set used for our regression. Looking at the results, we can see that for the training set, there is no significant Alpha, and the Beta is also not significant. Looking at the T-stat for Alpha, we also see that it is less than 2, so it is not significant. For our testing set, the Alpha is more significant but still not very significant. The Beta is significant in this testing set, and the T-stat is also more than it was in the training set, but it is still not significant. Looking at the adjusted Square value for both the testing and training sets, we see that it is very low, which means that the benchmark explains almost none of your returns. What does this mean, and why are the training set and testing set so different? The reason they are so different is essentially that random chance, the testing set just had data that the strategy was able to pick up on better compared to the training set,

but the reason the results are so poor in both of them is that our data has no structure. After all, it is randomly generated. Because our data is random, our strategy does not generate real Alpha, is not consistently related to the market, and occasionally picks up random trends, and this is what is generating our returns in the graph. Note that this model is also constructed before accounting for t costs. Making a model that uses the data after t costs are accounted for would just show worse results and may not show us what is actually going on in the data and the strategy. Next, we'll move on to the real data and see how that handles with our strategies.

2. Real Data

Given below is the heat plot for the optimal Sharpe ratio lookback periods for Bitcoin. We did not find a heat plot for all the different coins we used, but went through a couple of them, and they all have similar graphs to Bitcoin. Looking at the graph, we can see that the optimal lookback period is around 37 for X and 13 for Y, and for the short side, it is around 13 for N and 37 for M. The reason we are including this here is to show that the optimal look back. If the lookback is too small, it will result in more trading and higher turnover, which will increase the transaction costs.

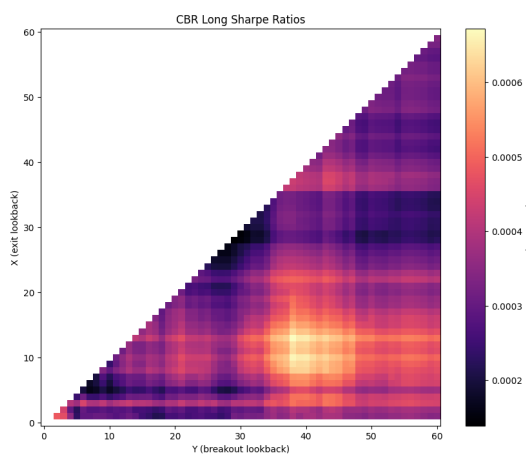


FIGURE 0.0.28. Training Set

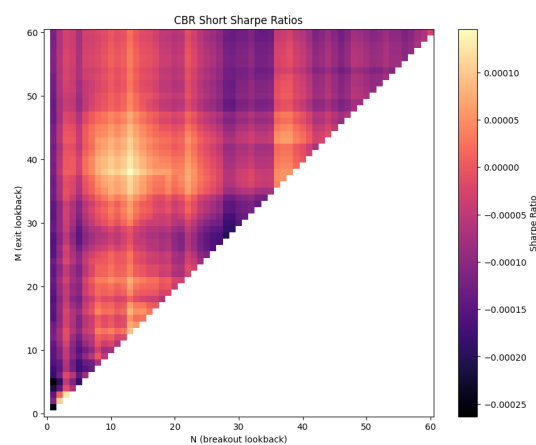


FIGURE 0.0.29. Testing Set

FIGURE 0.0.30.

Now that we're on the real data, we also need to calculate the weights and resample periodically because our data is time-stamped, and we can use the resample function in Python. We tried a couple of different recycling periods, like quarterly, monthly, and weekly, but we found that rebalancing daily worked the best for the momentum breakout strategy. This is likely due to the look-back period in the strategy being short; it could also be due to the fact that we are using 4-hour data. Once we find our portfolio with rebalancing as we go, we can plot the returns of the coins and the plot of the overall portfolio, which are given below.

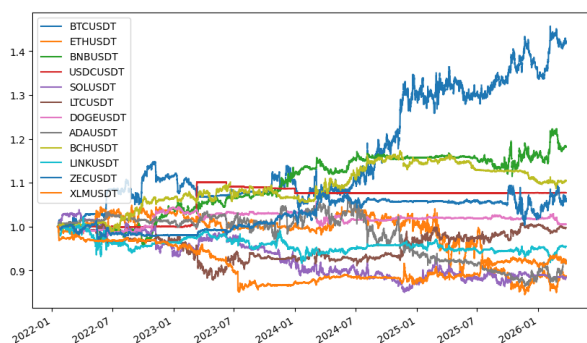


FIGURE 0.0.31. Coins

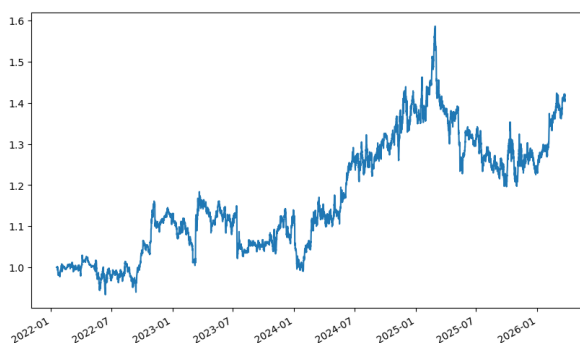


FIGURE 0.0.32. Portfolio

FIGURE 0.0.33.

Looking at the two plots, we can see that our strategy is working and is generating returns, but this is before we add in the costs, which we will do in the next step. Looking at the individual coins, we can see that some perform poorly, others perform okay, and some perform well. Looking at the overall portfolio, we can also see that it performed good as well.

Now we will apply the transaction cost and slippage to our strategy, plotting the portfolio, and we see that we also continuously lose money over time. This is not ideal, but the cause of this is probably due to our high turnover due to the short look-back periods in our strategy, making it not profitable to see what's going on. We will develop a model in the next part.

Below is the regression model for the training set the testing set had similar results with the Alpha and Beta but the Alpha T-stat was not as significant as the training said this could

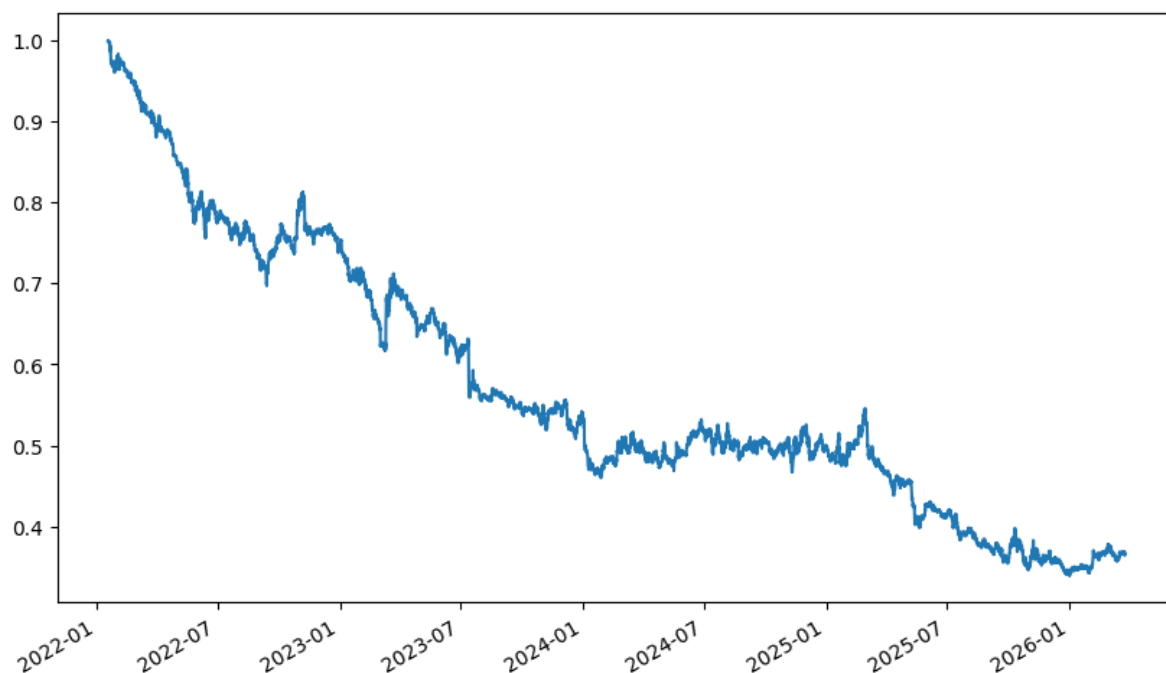


FIGURE 0.0.34.

be because we have 5 years worth of crypto data and the training model is on the first four years and the testing model is on the last year and 2025 through 2026 was weird for crypto. Regardless, we can see from the training set that the R^2 is very low, which indicates that the strategy is completely uncorrelated with the benchmark and the benchmark explains none of the returns. The small Beta indicates that our strategy is less sensitive than the overall Market. The Alpha is very close to zero, which means that there is very little access to returns that are not explained by the benchmark. The Alpha T-stat is not significant, which may indicate that our strategy does not outperform the expected return of the benchmark.

OLS Regression Results						
=====						
Dep. Variable:	y	R-squared:	0.002			
Model:	OLS	Adj. R-squared:	0.002			
Method:	Least Squares	F-statistic:	28.57			
Date:	Wed, 25 Mar 2026	Prob (F-statistic):	9.18e-08			
Time:	14:06:29	Log-Likelihood:	65216.			
No. Observations:	14697	AIC:	-1.304e+05			
Df Residuals:	14695	BIC:	-1.304e+05			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	2.627e-05	2.36e-05	1.113	0.266	-2e-05	7.26e-05
BTCUSD	0.0113	0.002	5.345	0.000	0.007	0.015
=====						
Omnibus:	4171.024	Durbin-Watson:	2.002			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	502173.298			
Skew:	-0.164	Prob(JB):	0.00			
Kurtosis:	31.634	Cond. No.	89.7			
=====						

FIGURE 0.0.35.

Now we will generate our model after accounting for transaction costs to see how much they change. We didn't do this in artificial data because we expected to have fairly poor results before transaction costs, which was the case. Although we did have poor results with the breakout strategy on the real data, we will still look at the regression model after transaction costs to see what will change, although we can expect the results to be worse. Below is the regression model of the training set after transaction costs are applied. Looking at the Alpha, we can see that it is negative, which means that we are consistently losing money that is not explained by the benchmark, and the -2.437 Alpha T-stat indicates very statistically significant losses. This is expected before our Alpha was slightly positive, but after accounting for transaction costs, we take on losses. The Beta is still very small, which indicates that the losses are coming from transaction costs and not bad market timing, because Beta is how much

of are returns move with the market. Because our Beta is close to zero, this indicates that our returns or losses are not tied to the market, which indicates that it is due to the transaction costs and slippage.

OLS Regression Results						
=====						
Dep. Variable:	strategy		R-squared:	0.002		
Model:	OLS		Adj. R-squared:	0.002		
Method:	Least Squares		F-statistic:	13.69		
Date:	Wed, 25 Mar 2026		Prob (F-statistic):	0.000217		
Time:	14:06:29		Log-Likelihood:	29982.		
No. Observations:	7330		AIC:	-5.996e+04		
Df Residuals:	7328		BIC:	-5.995e+04		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-0.0001	4.73e-05	-2.437	0.015	-0.000	-2.26e-05
benchmark	0.0157	0.004	3.700	0.000	0.007	0.024
=====						
Omnibus:	1786.227		Durbin-Watson:	2.081		
Prob(Omnibus):	0.000		Jarque-Bera (JB):	69208.215		
Skew:	-0.423		Prob(JB):	0.00		
Kurtosis:	18.030		Cond. No.	89.6		
=====						

FIGURE 0.0.36.

Our strategy did not turn out profitable after accounting for transaction costs and slippage, which is okay, but disappointing. We will move on to the exponential moving average to see if it does better with the live data and analyze it with regression models as well in the next step.

Now we will move on to the exponential moving average strategy. We will apply a rebalance when calculating the weights. We found that rebalancing quarterly worked very well for the exponential moving average strategy, and after messing around a little bit more, rebalancing every 100-days seems to have the best results. In part one, the EMA did not do as well, but rebalancing the weights helped this strategy a lot. Once we have this, we can plot the individual coins and see the returns. They are good for some coins, bad for others, and break even on some. We also combine this into a single portfolio to look at the overall returns, which is given below as well, and we can see it performs well until about mid 2024, where we start moving more horizontally. The first of the three graphs below is the returns of our exponential moving average strategy before t costs are applied. The second graph below is the returns of our strategy before t costs are applied to the portfolio. The third and final graph is the returns of the portfolio after t costs are applied. As you can see, they look very similar, but looking at the y-axis, we can see that everything has essentially shifted down slightly for our returns, which is expected, but our strategy is still profitable after t costs are applied with our exponential moving average strategy, which is what we were looking for.

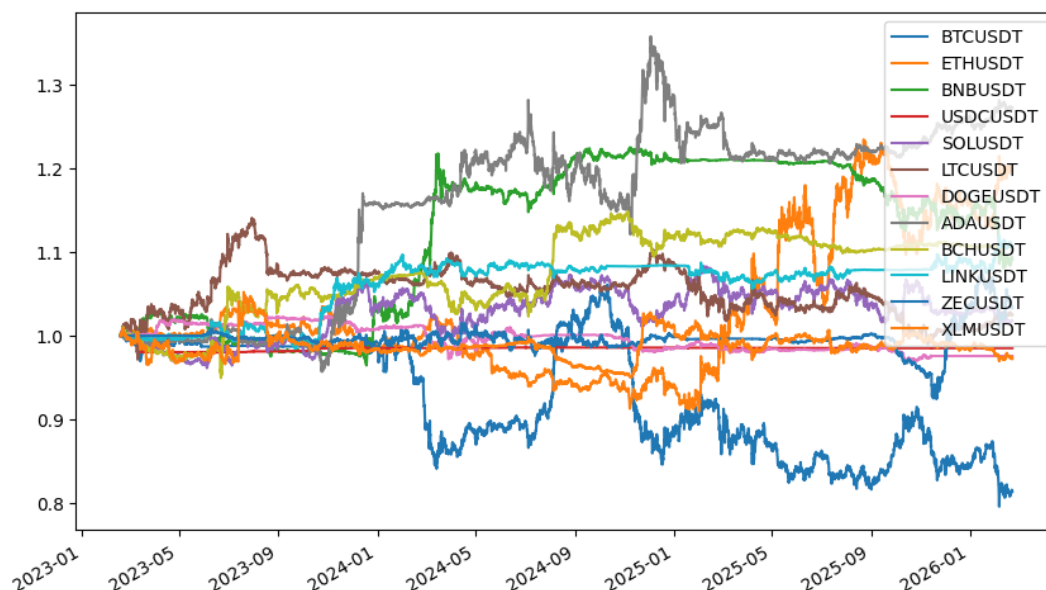


FIGURE 0.0.37.

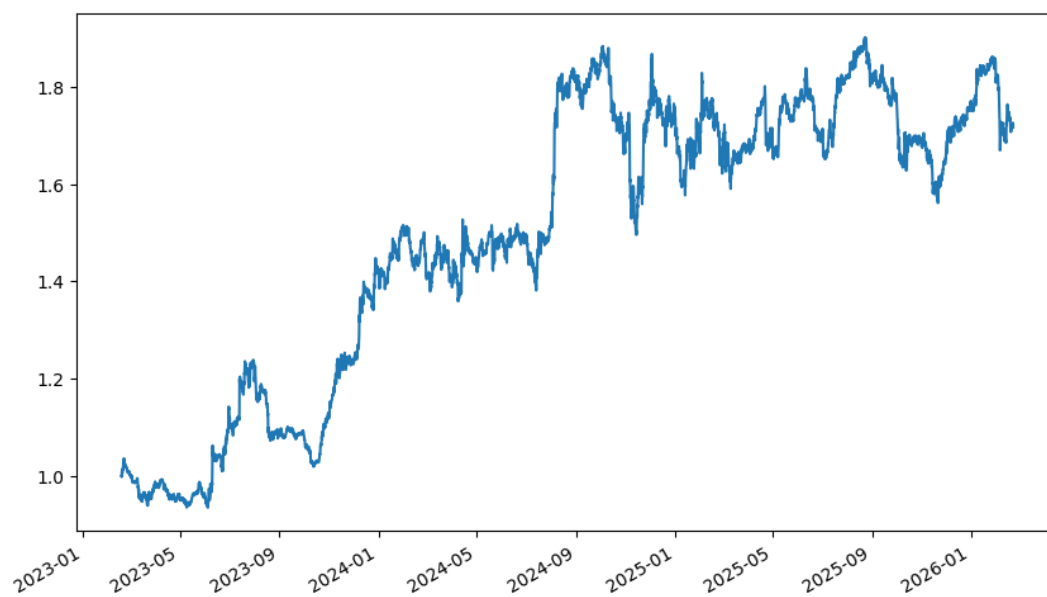


FIGURE 0.0.38.

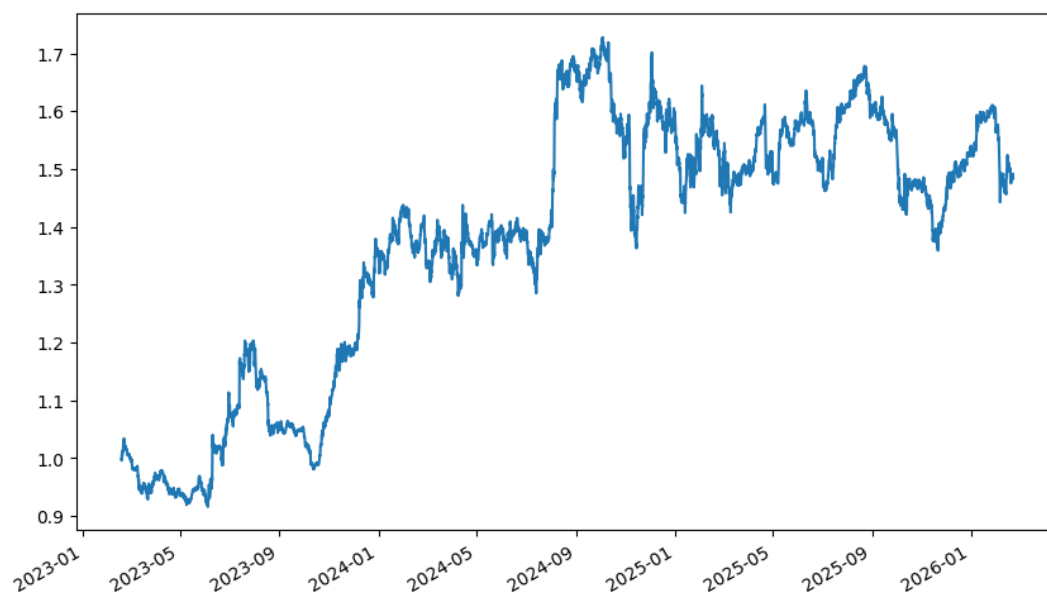


FIGURE 0.0.39.

Now we will move on to the regression models for our exponential moving average strategy. Below is the model for the training set. Looking at the Beta, we see that it is small, which indicates that it is not significant and suggests that our EMA strategy is not just following the market, and then our returns are coming from something else. Looking at our Alpha, we see that it is positive and close to zero, and it is statistically significant from the T-stat. This suggests that there is a real edge. Although a bigger Alpha would be ideal, this isn't bad; it still shows that our strategy is working, but that the signal is noisy. This is due to the volatility of crypto. The R-squared value is also close to zero, which tells us that the returns are not explained by the benchmark, so we are generating independent returns despite the low Alpha. The results for the testing set are similar for the Alpha and Beta; the T-stat for the Alpha was smaller, but the test set is using the last year of data, which did not perform as well, which could be the cause.

OLS Regression Results						
=====						
Dep. Variable:	strategy		R-squared:	0.004		
Model:	OLS		Adj. R-squared:	0.004		
Method:	Least Squares		F-statistic:	22.36		
Date:	Wed, 25 Mar 2026		Prob (F-statistic):	2.31e-06		
Time:	14:06:30		Log-Likelihood:	20439.		
No. Observations:	5236		AIC:	-4.087e+04		
Df Residuals:	5234		BIC:	-4.086e+04		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	0.0001	6.75e-05	2.047	0.041	5.85e-06	0.000
benchmark	-0.0316	0.007	-4.729	0.000	-0.045	-0.019
=====						
Omnibus:	1447.045		Durbin-Watson:	1.986		
Prob(Omnibus):	0.000		Jarque-Bera (JB):	53667.629		
Skew:	0.624		Prob(JB):	0.00		
Kurtosis:	18.634		Cond. No.	99.1		
=====						

FIGURE 0.0.40.

Now we'll move on to the next regression model. This model takes in the transaction cost and the slippage, whereas the previous model was before the transaction cost and slippage were applied. Below is the model for our strategy after the transaction costs are applied, and we see that the Alpha stayed the same, the Alpha T-stat decreased, and the Beta basically stayed the same. The R-squared value barely changes as well. The Alpha stayed the same, which indicates that we still have a real edge after t cost, it did not destroy it, and we are still profitable. The Alpha T-stat dropped, which means that it is less statistically significant after the t costs are applied, but there is still a signal, although it is weaker relative to noise. Beta is still small, which shows that we are not dependent on the benchmark and are still generating independent returns. The R-squared value is also small as well, which means that the benchmark explains none of our returns.

The testing set for the regression model had a positive Alpha that was very close to zero, and it also had a positive Beta that was close to zero as well, and a less statistically significant Alpha T-stat. Again, this is probably because the testing set uses the last year of data, which is mid 2025 to the present, which was bad for crypto, and our strategy also did not perform as well during that time period, which is why it is resulting in bad results. Having more data or using a different testing set may result in a better model for the testing set.

Although our breakout strategy was not profitable or exponential moving average strategy was still profitable with the real data after costs were applied, which is what we were looking for. Although it wasn't as significant as we wanted it to be, there is still merit in it, and it could be tradable on the real market.

OLS Regression Results						
Dep. Variable:	strategy	R-squared:	0.005			
Model:	OLS	Adj. R-squared:	0.004			
Method:	Least Squares	F-statistic:	24.46			
Date:	Wed, 25 Mar 2026	Prob (F-statistic):	7.82e-07			
Time:	14:06:30	Log-Likelihood:	20605.			
No. Observations:	5280	AIC:	-4.121e+04			
Df Residuals:	5278	BIC:	-4.119e+04			
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	0.0001	6.73e-05	1.576	0.115	-2.59e-05	0.000
benchmark	-0.0330	0.007	-4.946	0.000	-0.046	-0.020
Omnibus:	1439.297	Durbin-Watson:	1.991			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	51926.862			
Skew:	0.613	Prob(JB):	0.00			
Kurtosis:	18.314	Cond. No.	99.3			

FIGURE 0.0.41.

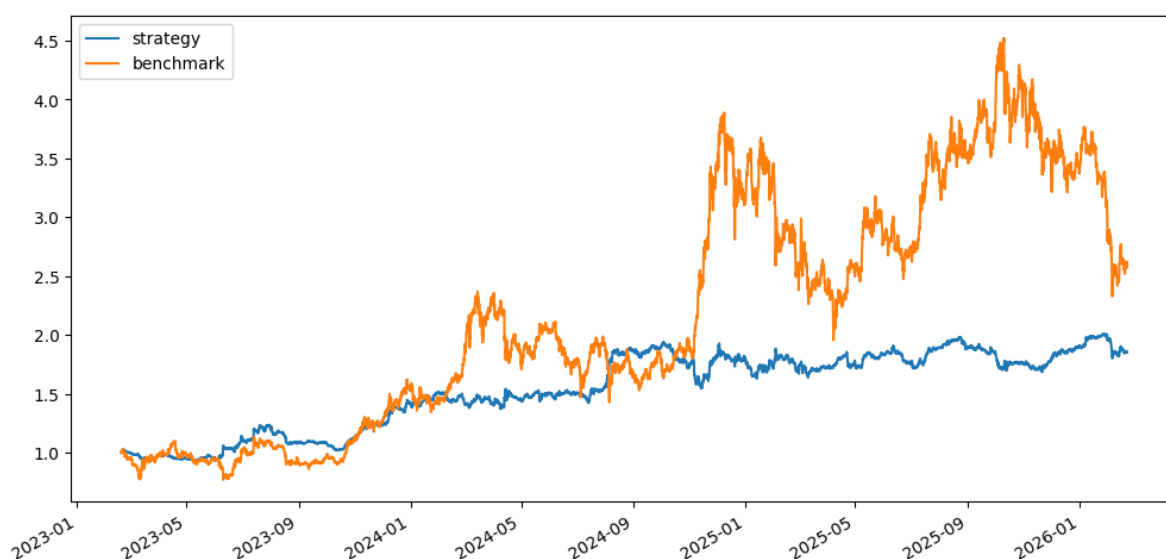


FIGURE 0.0.42.

Given above is a plot of the exponential moving average strategy versus our benchmark. We can see that the exponential moving average strategy is much less volatile than the benchmark, although the benchmark had higher returns in the end. This shows that our strategy works. Although it does not produce as many returns as the benchmark, it is much less volatile and could be leveraged to generate more returns than the benchmark, making it a profitable strategy.

3. Conclusion

In Part 2, the strategies developed earlier were evaluated using a more realistic framework that incorporated out of sample testing, rolling optimization, benchmark comparison, and transaction costs. These additions were critical in assessing whether the strategies could generalize beyond the data on which they were initially developed.

The results highlight a key distinction between statistical performance and economic viability. While both the breakout and exponential moving average (EMA) strategies showed some level of effectiveness on pre-cost artificial and real data, their performance diverged significantly once transaction costs were introduced. The breakout strategy, despite exhibiting weak positive Alpha before costs, generated a strongly negative and statistically significant Alpha after costs. This indicates that its high turnover led to trading costs overwhelming any predictive signal, rendering the strategy unprofitable in practice.

In contrast, the EMA strategy demonstrated greater robustness. Although its Alpha was relatively small and significant before costs, it remained slightly positive after transaction costs were applied, but less significant. This suggests that the EMA strategy benefits from lower turnover and more selective trading, allowing it to retain its signal in a realistic setting. Additionally, both strategies exhibited near-zero Beta with respect to the benchmark, indicating that their returns were largely independent of overall market movements and instead driven by their internal logic.

A central takeaway from this analysis is that the relationship between signal strength and trading frequency is crucial. Strategies that trade too frequently may appear profitable in

theory but fail once t costs are included. Conversely, strategies with modest but consistent signals can remain viable if they control turnover effectively.

Overall, the findings emphasize that evaluating a strategy requires more than examining raw returns. Incorporating out of sample validation, transaction costs, and regression based performance metrics such as Alpha and its t -statistic provides a more accurate measure of whether a strategy is truly implementable. In this case, the EMA strategy shows potential as a tradable approach, while the breakout strategy does not generalize well under realistic conditions.

REFERENCES

- [1] Cowles, A. (n.d.). *Efficient-market hypothesis*. Wikipedia, the free encyclopedia. Retrieved September 6, 2025, from https://en.wikipedia.org/wiki/Efficient-market_hypothesis
- [2] Random walk hypothesis. (n.d.). *Random walk hypothesis*. Wikipedia, the free encyclopedia. Retrieved September 6, 2025, from https://en.wikipedia.org/wiki/Random_walk_hypothesis
- [3] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Penn Arts & Sciences. Retrieved August 19, 2025, from <https://www.sas.upenn.edu/~fdiebold/NoHesitations/BookAdvanced.pdf>
- [4] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. UC Berkeley Statistics Department. Retrieved August 19, 2025, from https://www.stat.berkeley.edu/~rabbee/s154/ISLR_First_Printing.pdf
- [5] Wikipedia contributors. (2026). *Alpha (finance)*. Retrieved March 19, 2026, from [https://en.wikipedia.org/wiki/Alpha_\(finance\)](https://en.wikipedia.org/wiki/Alpha_(finance))
- [6] Wikipedia contributors. (2026). *Beta (finance)*. Retrieved March 19, 2026, from [https://en.wikipedia.org/wiki/Beta_\(finance\)](https://en.wikipedia.org/wiki/Beta_(finance))
- [7] Wikipedia contributors. (2026). *Fama–French three-factor model*. Retrieved March 19, 2026, from https://en.wikipedia.org/wiki/Fama%E2%80%93French_three-factor_model
- [8] Chen, K., & Chen, J. (2026). *A Forecast Model for Crypto Currencies*. Available at SSRN: <https://ssrn.com/abstract=6300843>. <http://dx.doi.org/10.2139/ssrn.6300843>
- [9] Unknown author. (n.d.). *Algorithmic trading strategy model based on exponential moving average for Cryptocurrencies*. Retrieved March 19, 2026, from <http://bib.pucminas.br:8080/pergamumweb/vinculos/00000e/00000e0c.pdf>
- [10] Boyd, S., et al. (n.d.). *Crypto Portfolio Optimization*. Retrieved March 19, 2026, from <https://web.stanford.edu/~boyd/papers/pdf/crypto-portfolio.pdf>
- [11] Python Software Foundation. (2026). *Python 3.14.3 Documentation*. Retrieved March 19, 2026, from <https://docs.python.org/3/>
- [12] Asabeneh, A. (n.d.). *30 Days of Python: A step-by-step guide to learn Python programming*. GitHub. Retrieved March 19, 2026, from <https://github.com/Asabeneh/30-Days-Of-Python>
- [13] Khamis Hamed Al-Yahyaee, Walid Mensi, Hee-Un Ko, Seong-Min Yoon, Sang Hoon Kang. *Why cryptocurrency markets are inefficient: The impact of liquidity and volatility*. sciencedirect. Retrieved March 19, 2026, from <https://www.sciencedirect.com/science/article/abs/pii/S1062940820300656>

Here is the source code for the project available on my GitHub

<https://github.com/davidjackson2192112-glitch/Code/blob/main/Quant%20Project.ipynb>